eURC

eURC Point of Sale (React + Node.js) · Hands-on Project 02 · Application architecture

Logical architecture — React SPA, Node.js API, Azure data services

Sheet 1 · components, layers, dependencies

Cashier
Register · Sales

Store manager
+ Products · Report · Staff

HTTPS (TLS 1.2+)

Browser · React 19 single-page app (web/, built by Vite 8)

App.jsx — shell

Header: store, user, **environment badge** (DEV / TEST / PROD, version, slot) from /api/info
Tabs by role · session in sessionStorage · 401 → back to sign-in

Register.jsx

Product grid, search, category chips
adds to cart (stock-aware)

Login.jsx

Sign in (store code, e-mail, password)
Create a store (becomes manager)

Cart.jsx

lines, qty ±
subtotal, tax, total (preview)

CheckoutDialog

cash: tendered, quick amounts, change · card: terminal code

Receipt.jsx

printable 80 mm receipt · refund stamp

Manager screens

Products.jsx catalogue, price, tax, receive stock, hide · **Sales.jsx** day list, receipt, refund
Report.jsx gross, transactions, average, tax, cash / card, top products · **Team.jsx** staff, roles, deactivate

lib/

api.js fetch wrapper: same origin, Bearer token, JSON errors
money.js cents maths (mirror of API pricing), CAD format, cash parsing, quick tenders
useLoader.js data loading hook (reload, 401 handling) · **dates.js** local day + tz offset

Quality (web/)

ESLint + react-hooks rules · **Vitest + Testing Library**: money, Cart, CheckoutDialog, Login, whole App against a fake API (17 tests, ≥ 80 % lines) · Playwright E2E (tests/) drives the real UI in the test environment on desktop + tablet

No secrets in the browser. The SPA only holds the signed session token for the tab (sessionStorage, 8 h). Prices, totals, tax, stock and roles are always decided by the API.

Build: `vite build` → web/dist → copied to app/public in the CD build job → one package.

HTTPS
JSON

Azure App Service · Node.js 22 + Express 5 (app/) — one process serves the UI and the API

Middleware pipeline (every request)

① security headers: HSTS, **CSP**, nosniff, frame DENY, X-EURC-Version / Environment ② JSON body ≤ 128 kB ③ rate limiting on POST /api/stores and /api/auth/login ④ **auth**: Bearer token → HMAC verify (key from Key Vault, re-read on rotation) → user still active? ⑤ **managerOnly** for manager routes ⑥ error handler: 4xx messages, 5xx logged only

every route below passes through the middleware

/health · /health/live

SQL, Blob, Key Vault probes; schema version; 503 while migrating

/api/info

version, commit, env, slot

/api/stores · /api/auth

sign-up, sign-in, /api/me; DELETE store (tests clean-up)

/api/users

staff list / add / deactivate (manager)

/api/products

catalogue, search, price, stock

/api/sales

checkout, list, receipt, refund

/api/reports/daily

totals by payment, top products

static + SPA fallback

React build, deep links → index.html

Domain (src/domain/)

pricing.js integer cents · tax in basis points, rounded half-up per line · basket validation, merge duplicates, ≤ 999 per line · cash ≥ total → change · card → terminal reference only
auth.js scrypt password hashes · HMAC-SHA256 tokens, 8 h, tamper-proof
migrations.js versioned, expand-only; `validateMigrations` rejects DROP / rename

Adapters (dependency-injected)

azure.js — production
• mssql pool, **token-credential** (managed identity)
• transactions, UPDLOCK receipt numbers, sp_getapplock for migrations
• BlobServiceClient · SecretClient
memory.js — same contract, for local dev + unit tests

Start-up (server.js)

listen → apply pending migrations under app lock → ready (health 200)
retry with back-off · graceful SIGTERM · telemetry first (OpenTelemetry → App Insights)

Configuration (app settings)

EURC_ENVIRONMENT · EURC_SLOT (sticky) · AZURE_CLIENT_ID · SQL_SERVER / SQL_DATABASE · STORAGE_BLOB_URL · KEY_VAULT_URL · APPLICATIONINSIGHTS_CONNECTION_STRING — **no passwords or keys**

Quality (app/)

ESLint · **31 node:test tests** (domain maths, tokens, roles, checkout, all-or-nothing stock, refunds, report, store isolation, health failure modes) · coverage ≥ 80 % lines · npm audit · pipeline suites: smoke 8, integration 13, load gate

build.json (version, commit, run) travels in the package → shown by /api/info, the UI badge and every log line.

Azure data services (per environment)

id-eurc-app-<env>-cc
user-assigned, both slots

Azure SQL (T-SQL over TDS) sqldb-eurc-core-<env>
stores · users · products · sales · sale_lines · schema_migrations
Entra-only auth (identity = Entra admin)

Blob Storage container *pos-receipts*
receipts/<storeId>/<saleId>.json
rebuilt from SQL if a blob is missing

Key Vault secret *pos-token-signing-key*
created on first start; shared by all instances and both slots, so a swap keeps cashiers signed in

Application Insights (+ Log Analytics)
requests, dependencies (SQL, Blob, KV), exceptions, tagged with version, environment, slot

Resource	Role for the app identity
SQL server	Entra admin (migrations + data)
pos-receipts	Storage Blob Data Contributor
Key Vault	Key Vault Secrets Officer

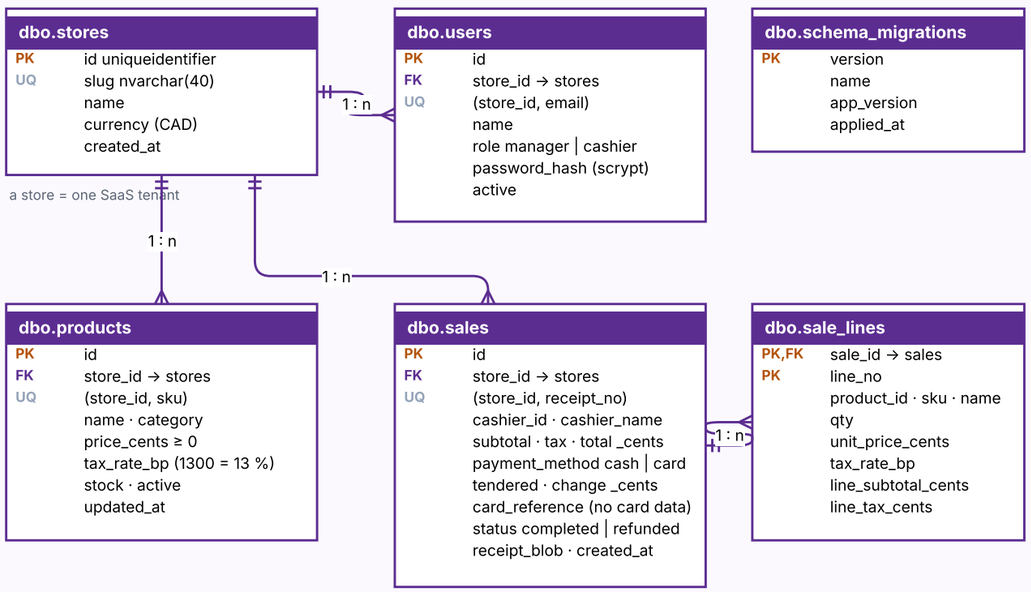
In prod all three are reached through private endpoints from the VNet-integrated App Service (see the Azure infrastructure drawing).

HTTPS from the browser
in-process call
managed-identity access
telemetry

Project: **eURC POS · P02**	Drawing: **EURC-P02-APP-01**	Sheet: **1 of 2**
Revision: **1.1 · 2026-10-23**	Standard: **Azure CAF naming**	Format: **A3 landscape**

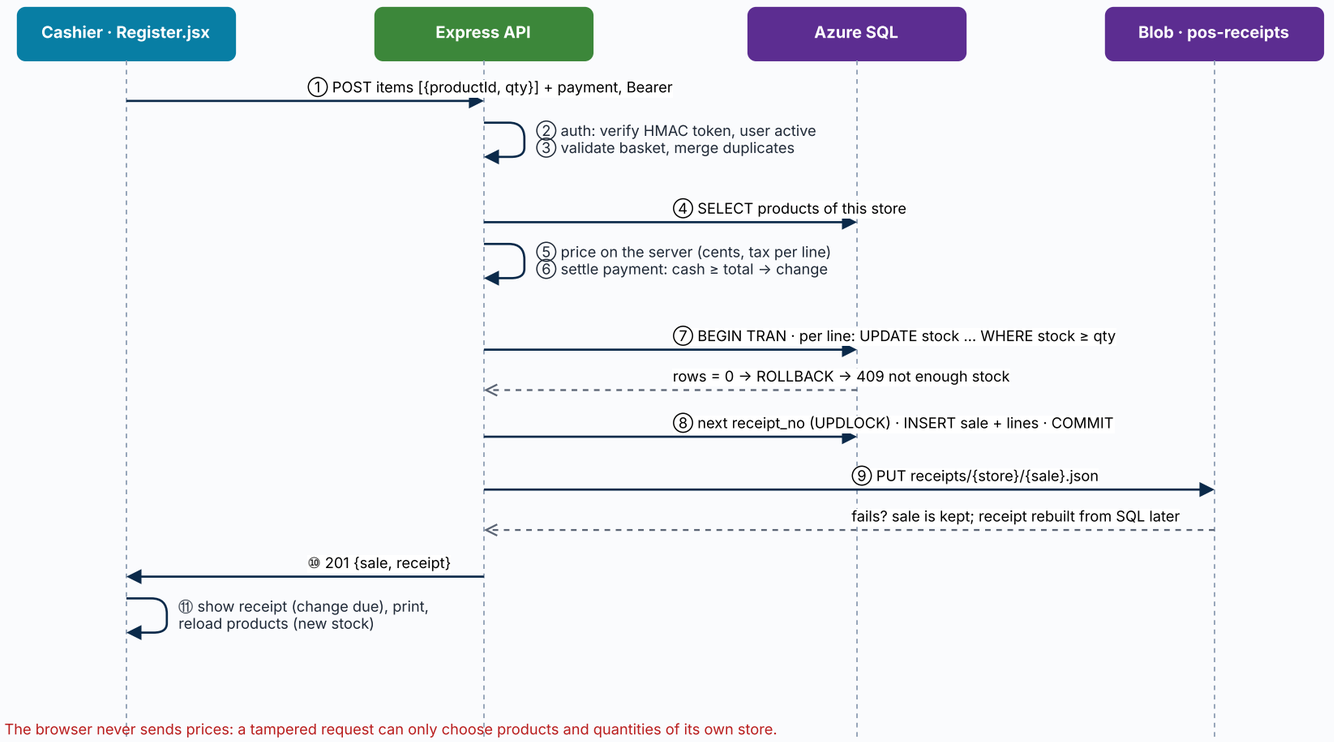
Data model, API and key flows

A · Data model (Azure SQL, migrations 1–3; money in integer cents)



Deleting a store cascades to all its rows; sale lines copy SKU, name, price and tax so receipts never change when the catalogue does. Index ix_sales_store_created serves the day list and report.

B · Checkout flow (POST /api/sales) — all-or-nothing



The browser never sends prices: a tampered request can only choose products and quantities of its own store.

C · REST API (JSON over HTTPS)

Method	Path	Who	Purpose
GET	/health · /health/live	public	readiness (SQL, Blob, KV, schema) · liveness
GET	/api/info	public	version, commit, environment, slot
POST	/api/stores	public, rate-limited	create store + manager → session
POST	/api/auth/login	public, rate-limited	store code + e-mail + password → session
GET	/api/me	signed in	current user and store
GET · POST · PATCH	/api/users[/:id]	manager	list, add, (de)activate staff
GET	/api/products?q=&all=1	signed in	catalogue (all=1: hidden too, manager)
POST · PATCH	/api/products[/:id]	manager	add, edit price/tax/stock, hide
POST	/api/sales	signed in	checkout (flow B)
GET	/api/sales?date=&tz=	signed in	sales of a local day
GET	/api/sales/:id[/receipt]	signed in	sale · receipt (from Blob)
POST	/api/sales/:id/refund	manager	refund + restock (once)
GET	/api/reports/daily?date=&tz=	manager	gross, tax, count, average, cash/card, top 5
DELETE	/api/stores/current	manager + confirm	delete store, rows and receipts

D · Sign-in, sessions and roles

Sign-in

1. POST /api/auth/login {store, email, password}
2. find store by code → user by e-mail
3. scrypt verify (constant time); same 401 for any mistake
4. token = base64url(claims) . HMAC-SHA256(key)
   claims: store id, user id, role, expiry 8 h
   key: Key Vault *pos-token-signing-key*
5. every request: verify signature + expiry, reload user (deactivation is immediate), scope all queries to the store

Capability	Cashier	Manager
Ring up sales (cash / card)	✓	✓
View sales and receipts	✓	✓
Refund a sale	–	✓
Products, prices, stock	–	✓
Daily report	–	✓
Staff and roles	–	✓

Roles are enforced by the API (403); the UI only hides tabs.

E · Pricing rules — worked example

Line	Subtotal	Tax
2 × Coffee $2.45 (13 %)	$4.90	$0.637 → **$0.64**
1 × Muffin $2.95 (0 %)	$2.95	$0.00
Total	$7.85	**$8.49** due
Cash $10.00		change **$1.51**

- integer cents end to end (no floating-point money)
- tax per line, rounded half-up, in basis points
- same rule in API (pricing.js) and UI preview (money.js), both unit-tested with this basket
- the same basket is the tester's integration and E2E oracle

Project **eURC POS · P02**	Drawing **EURC-P02-APP-02**	Sheet **2 of 2**
Revision **1.1 · 2026-10-23**	Standard **Azure CAF naming**	Format **A3 landscape**